

SVEUČILIŠTE U SPLITU
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I BRODOGRADNJE

FORENZIČKA ANALIZA DIGITALNE SLIKE

Laboratorijske vježbe

Maja Braović / Damir Krstinić

Split, 2021.

ODRICANJE OD ODGOVORNOSTI

Informacije sadržane u ovome dokumentu dane su u edukativne svrhe, tj. u svrhu održavanja laboratorijskih vježbi iz kolegija Forenzička analiza digitalne slike na Fakultetu elektrotehnike, strojarstva i brodogradnje pri Sveučilištu u Splitu. Autori ovoga dokumenta nisu odgovorni za način na koji bi se informacije sadržane u njemu mogle koristiti, kao ni za posljedice koje pritom eventualno nastanu.

Autori

Sadržaj

1	Uvod u obradu i analizu digitalne slike	6
1.1	Osnovne operacije na slici	6
1.2	Promjena dimenzija slike	7
1.3	Prostori boja	8
1.4	Pristup pikselima slike	9
1.5	Histogrami	10
2	Vodeni žigovi	11
2.1	Postavljanje vodenog žiga na sliku	11
2.2	Kako dokazati vlasništvo nad digitalnom slikom?	12
2.3	Kako osigurati vodeni žig na slici?	13
3	Metapodaci	14
3.1	EXIF podaci	14
4	Provjera autentičnosti digitalne slike	18
4.1	Metapodaci	18
4.2	Checksum	18
4.3	Informacije o digitalnoj kameri	19
4.3.1	Provjera specifikacija kamere	19
4.3.2	PRNU otisak kamere	19
5	Steganografija	21
5.0.1	Implementacija steganografije u programskom jeziku Python	24
6	Detekcija krivotvorenih JPEG slika pomoću DCT transformacije	26
7	Detekcija krivotvorenih JPEG slika pomoću LBP algoritma	29
8	Antiforenzika	30
	Literatura	31

Popis oznaka i kratica

ASCII – American Standard Code for Information Interchange

DCT – Direct Cosine Transform

EXIF – Exchangeable Image File Format

GPS – Global Positioning System

ISO – International Organization for Standardization

IPTC – International Press Telecommunications Council

JPEG – Joint Photographic Experts Group

LBP – Local Binary Patterns

OpenCV – Open Source Computer Vision

PNG – Portable Network Graphics

PRNU – Photo Response Non Uniformity

RGB – Red Green Blue

XMP – Extensible Metadata Platform

Važne informacije prije početka laboratorijskih vježbi

Izv. prof. Damir Krstinić

- Ured: B502
- E-mail: damir.krstinic@fesb.hr
- Broj telefona: 305-895

Doc. dr. sc. Maja Braović

- Ured: B501
- E-mail: maja.braovic@fesb.hr
- Broj telefona: 305-601

Za svaku laboratorijsku vježbu potrebno je napisati izvještaj i postaviti ga na e-Learning portal. Vremensko ograničenje za predaju svih izvještaja je do kraja posljednjeg tjedna nastave.

Na svim laboratorijskim vježbama iz Digitalne obrade i analize slike koristiti će se programski jezik Python. Programi se u Pythonu pišu u datoteci s ekstenzijom .py, a pokreću se naredbom `python mojProgram.py` (za Python 2) ili `python3 mojProgram.py` (za Python 3).

U slučaju da neka od potrebnih Python biblioteka nije instalirana, možete je instalirati preko Command Prompt-a naredbom: `pip install ime-biblioteke`.

Budući da su neke laboratorijske vježbe složenije i duže od drugih, njih ćemo obrađivati duže (tj. u više termina) nego one kraće.

Da bi se laboratorijske vježbe položile potrebno ih je sve odraditi. U slučaju izostanka sa jedne laboratorijske vježbe ona se može nadoknaditi pri kraju semestra za vrijeme nadoknadi laboratorijskih vježbi. Osim u iznimnim slučajevima (npr. duža bolest koju je potrebno dokazati liječničkom ispričnicom, predstavljanje Fakulteta na studentskim natjecanjima i sl.), na nadoknadama se ne može nadoknaditi više od jedne laboratorijske vježbe.

1 Uvod u obradu i analizu digitalne slike

U ovoj laboratorijskoj vježbi upoznati ćemo se s osnovnom terminologijom koja se koristi prilikom obrade i analize digitalne slike u programskom jeziku Python.

1.1 Osnovne operacije na slici

Primjer učitavanja, prikaza i spremanja slike:

```
from PIL import Image

slika = Image.open("slika.jpg")
slika.show()
slika.save("sacuvana_slika.jpg")
```

Prikaz informacija o slici:

```
from PIL import Image

slika = Image.open("slika.jpg")

stupci, retci = slika.size

print("Broj stupaca: " + str(stupci))
print("Broj redaka: " + str(retci))

if (slika.mode == "RGB"):
    print("Broj kanala: 3")
else:
    print("Broj kanala: 1")
```

Zadatak 1. U Pythonu učitajte sliku u boji te ispišite njezin broj stupaca, redaka i kanala.

Primjer pretvorbe slike u boji u sliku u razinama sive boje:

```
from PIL import Image
from PIL import ImageOps

slika = Image.open("slika.jpg")
siva_slika = ImageOps.grayscale(slika)
siva_slika.save("siva_slika.jpg")
```

Zadatak 2. U Pythonu učitajte sliku u boji te je konvertirajte u sliku u razinama sive boje. Dobivenu sliku prikažite i spremite.

Primjer primjene različitih filtera na učitano sliku:

```
from PIL import Image
from PIL import ImageFilter

slika = Image.open("slika.jpg")

detektirani_rubovi = slika.filter(ImageFilter.FIND_EDGES)
zamucena_slika = slika.filter(ImageFilter.BLUR)
izostrena_slika = slika.filter(ImageFilter.SHARPEN)

detektirani_rubovi.save("detektirani_rubovi.jpg")
zamucena_slika.save("zamucena_slika.jpg")
izostrena_slika.save("izostrena_slika.jpg")
```

Zadatak 3. U Pythonu učitajte sliku u boji te primijenite na nju proizvoljni filter. Dobivenu sliku prikažite i spremite.

1.2 Promjena dimenzija slike

Primjer promjene dimenzija slike:

```
from PIL import Image
from PIL import ImageOps

slika = Image.open("slika.jpg")

stupci, retci = slika.size

# 1. način - specifikacija novog broja redaka i stupaca
smanjena_slika_1 = slika.resize((256, 256))

# 2. način - skaliranje (// - integer dijeljenje)
smanjena_slika_2 = slika.resize((stupci//2, retci//2))

smanjena_slika_1.save("smanjena_slika_1.jpg")
smanjena_slika_2.save("smanjena_slika_2.jpg")
```

Zadatak 4. U Pythonu učitajte sliku u boji te je umanjite na način da bude upola manja no u originalu.

1.3 Prostori boja

Prostor boja je matematički model koji definira određeni rang boja te time određuje način na koji se slika prikazuje. Najpoznatiji prostor boja je RGB (engl. *Red Green Blue*), no uz njega postoje i brojni drugi, na primjer HSV (engl. *Hue Saturation Value*), HSI (engl. *Hue Saturation Intensity*), CIE $L^*a^*b^*$ i CIE $L^*u^*v^*$.

Različiti prostori boja se koriste u automatskoj klasifikaciji i segmentaciji digitalne slike zbog toga što različite klase vidljive na slikama (npr. oblaci, more, vatra, snijeg) nisu jednako uočljivi u svim prostorima boja, tj. u nekim prostorima boja su uočljivije nego u drugima.

Slike prikazane pomoću RGB prostora boja sastoje se od 3 kanala: crvenog, zelenog i plavog. Slika 1.1 prikazuje rastav RGB slike na 3 kanala. Za ulaznu sliku na slici 1.1 korištena je slika iz baze slika Berkeley Segmentation Data Set (BSDS500) [1].



Slika 1.1: Slika prikazuje rastav RGB slike na tri kanala

Primjer rastavljanja RGB slike na kanale:

```
from PIL import Image

slika = Image.open("slika.jpg")

R, G, B = slika.split()

R.save("crveni_kanal.jpg")
G.save("zeleni_kanal.jpg")
B.save("plavi_kanal.jpg")
```

Zadatak 5. U Pythonu učitajte sliku u boji i rastavite je na kanale. Svaki kanal prikažite kao zasebnu sliku i spremite. Koji vam od pojedinačnih kanala izgleda najvjernije izvornoj slici?

1.4 Pristup pikselima slike

Primjer pristupanja pikselima slike te primjer promjene njihovih vrijednosti u Pythonu:

```
from PIL import Image

slika = Image.open("slika.jpg")

stupci, retci = slika.size

print("Broj stupaca: " + str(stupci))
print("Broj redaka: " + str(retci))

broj_kanala = 0

if (slika.mode == "RGB"):
    print("Broj kanala: 3")
    broj_kanala = 3
else:
    print("Broj kanala: 1")
    broj_kanala = 1

matrica_sa_pikselima = slika.load()

slika.show()

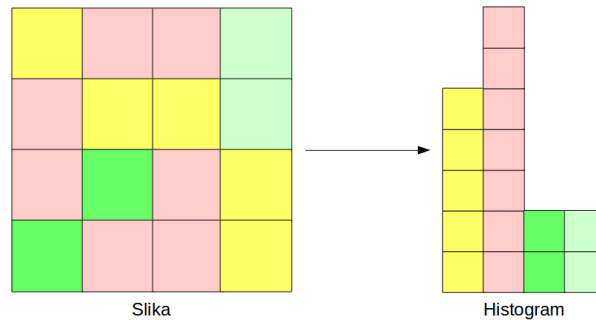
for i in range (0, stupci):
    for j in range (0, retci):
        if (matrica_sa_pikselima[i,j] != (0,0,0)):
            if (broj_kanala == 3):
                matrica_sa_pikselima[i,j] = (255,255,255)
            else:
                matrica_sa_pikselima[i,j] = (255,255)

slika.show()
```

Zadatak 6. U Pythonu učitajte sliku u boji te sve njezine piksele čiji je intenzitet u sva tri kanala veći od 100 promijenite u 255.

1.5 Histogrami

Histogram slike prikazuje broj piksela na slici po razinama intenziteta. Na primjer, ako se na slici nalazi 1000 piksela koji imaju intenzitet 32, onda će pripadajući stupac histograma imati visinu 1000. Shematski prikaz histograma dan je na slici 1.2.



Slika 1.2: *Shematski prikaz histograma*

Primjer računanja i prikaza histograma u Pythonu:

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
```

```
slika = Image.open("slika.png")
slika_grayscale = slika.convert('L')
slika_matrix = np.asarray(slika_grayscale)

plt.hist(slika_matrix.flatten())
plt.xlabel('Intenzitet')
plt.ylabel('Broj piksela')
plt.show()
```

Zadatak 7. U Pythonu učitajte sliku u razinama sive boje te prikažite njen histogram.

2 Vodeni žigovi

Vodeni žigovi se u obradi i analizi digitalnih slika koriste za prikrivanje informacija, te su stoga važni u forenzičkoj analizi digitalnih slika.

Vodeni žigovi (engl. *watermarks*) se koriste za stavljanje pečata na digitalne slike. Svrha stavljanja vodenih žigova je zaštita digitalne slike, tj. osiguravanje slike od krađe i povrede autorskih prava. Iako su neki vodeni žigovi uočljivi na slikama, postoje i neprimjetni vodeni žigovi kojih je jako teško (ili gotovo nemoguće) uočiti golim okom.

2.1 Postavljanje vodenog žiga na sliku

Na slici 2.1 možete vidjeti primjer vodenog žiga. Gornje dvije slike na toj slici predstavljaju originalnu sliku i sliku koja će postati videni žig na originalnoj slici. Donja slika predstavlja originalnu sliku na koju je postavljen vodeni žig.



Slika 2.1: *Primjer vodenog žiga*

Primjer postavljanja vodenog žiga na sliku:

```
from PIL import Image

slika = Image.open("slika.png")
watermark = Image.open("watermark.png")

skala = 0.1
Image.blend(slika, watermark, skala).save("slika_sa_watermarkom.png")
```

Zadatak 1. U Pythonu učitajte dvije različite slike istih dimenzija. Prva slika neka predstavlja originalnu sliku, a druga slika vodeni žig koji će se primijeniti na originalnu sliku. Postavite vodeni žig na sliku. Što se sa rezultatnom slikom događa kada mijenjate vrijednost skale?

2.2 Kako dokazati vlasništvo nad digitalnom slikom?

Na Internetu nažalost često možemo vidjeti primjere neovlaštene upotrebe tuđeg intelektualnog vlasništva, npr. digitalnih slika. Zbog ovog problema mnoštvo slika na Internetu zaštićeno je vodenim žigovima koji mogu biti vidljivi ili nevidljivi golim okom. Ako se dogodi da netko neovlašteno koristi vašu digitalnu sliku (koju ste vi ranije zaštitili vodenim žigom), možete to jednostavno dokazati ako imate u posjednu originalnu sliku bez vodenog žiga. Oduzimanje slike sa vodenim žigom od iste te slike prije no što je na nju primijenjen vodeni žig rezultirati će otkrivanjem vodenog žiga. Na primjer, otkrivanje vodenog žiga za sliku 2.1 dan je na slici 2.2. Prvi dio slike 2.2 predstavlja razliku između originalne slike i one na kojoj se nalazi vodeni žig, a drugi dio slike predstavlja istu tu razliku sa pojačanim kontrastom (budući da obična razlika ponekad nije vidljiva golim okom).



Slika 2.2: *Primjer otkrivenog vodenog žiga (bez i sa pojačanjem kontrasta)*

Primjer otkrivanja vodenog žiga na digitalnoj slici:

```

from PIL import Image
from PIL import ImageChops
from PIL import ImageEnhance

slika = Image.open("slika.png")
watermark = Image.open("watermark.png")

razlika = ImageChops.difference(slika, watermark)

razlika.save("razlika.png")

# Pojačajmo razliku da bude vidljivija
pojacana_razlika = ImageEnhance.Contrast(razlika)
skala = 7
pojacana_razlika_slika = pojacana_razlika.enhance(skala)

pojacana_razlika_slika.save("pojacana_razlika.png")

```

Zadatak 2. Otkrijte i pojačajte vodeni žig iz prvog zadatka. Što se događa sa rezultantom slikom kada mijenjate vrijednost skale?

2.3 Kako osigurati vodeni žig na slici?

Budući da se vodene žigovi uz dovoljno vremena i strpljenja često mogu ukloniti sa digitalne slike, u praksi postoji nekoliko smjernica pomoću kojih se taj proces otežava [2]:

- vodeni žig postavite iznad područja slike koje je ispunjeno detaljima, jer će ga biti teže ukloniti u programima za obradu digitalnih slika,
- neka boja vodenog žiga bude slična boji pozadine,
- neka vodeni žig zauzima što veću površinu na slici,
- neka vodeni žig bude što složeniji (tj. sa što više detalja),
- neka vodeni žig bude ili što prozirniji (tj. neprimjetan) ili što neprozirniji (tako da se ne vidi što se iza njega nalazi na slici).

Zadatak 3. Koristeći se gornjim savjetima, konstruirajte vodeni žig kojega će biti teško ukloniti sa slike te ga postavite na sliku.

3 Metapodaci

Metapodaci (engl. *metadata*) su dodatne informacije o digitalnoj slici koje slika može sadržavati uz podatke o boji piksela. Metapodaci obično sadrže informacije o uređaju kojim je slika snimljena te o postavkama tog uređaja prilikom snimanja te slike.

Primjeri metapodataka su:

- dimenzije slike, tj. broj stupaca, redaka i kanala,
- GPS koordinate na kojima je slika nastala,
- ISO razina korištena prilikom snimanja slike,
- informacija o tome je li blic bio korišten prilikom snimanja slike,
- ime programa u kojemu je slika posljednji put obrađena,
- koliki je bio otvor blende prilikom snimanja slike,
- model leće fotoaparata korištenog za snimanje slike.

Ako se digitalna slika promijeni, promijeniti će se i njezini metapodaci, što je izuzetno važno u forenzici jer forenzičarima omogućava ispitivanje autentičnosti i integriteta slike. Zanimljive informacije o integritetu digitalnih slika možete pronaći u [3].

Najpoznatiji standard za metapodatake koji se koristi kod digitalnih slika je EXIF (engl. *Exchangeable Image File Format*), no postoje i drugi poput IPTC (engl. *International Press Telecommunications Council*) Information Interchange Model i XMP (engl. *Extensible Metadata Platform*) standarda.

3.1 EXIF podaci

EXIF (engl. *Exchangeable Image File Format*) podaci su metapodaci koji se vežu uz sliku koja je snimljena digitalnim fotoaparatom, mobitelom, tabletom, i sl., te sačuvana u jednom od različitih formata za zapis digitalnih slika koje podržavaju EXIF standard (npr. JPEG i TIFF, no ne i PNG).

Kada se digitalne fotografije postavljaju na Internet, EXIF podaci mogu ostati vezani uz njih. Ako ti podaci sadrže informacije o GPS koordinatama na kojima je slika snimljena, mogu predstavljati veliki sigurnosni rizik za onoga tko je tu sliku snimio, jer se može otkriti njegova ili njezina lokacija. **Uvijek je preporučljivo provjeriti EXIF podatke slike koju želite postaviti na Internet, te ih izbrisati u slučaju da ne želite da budu dostupne svima.** U [4]

Basic	Permissions	Open With	Image
Image Type		jpeg (JPEG)	
Width		768 pixels	
Height		512 pixels	
Camera Brand		Canon	
Camera Model		Canon EOS 7D	
Date Taken		2014:03:30 10:56:46	
Exposure Time		1/250 sec.	
Aperture Value		6,00 EV (f/8,0)	
ISO Speed Rating		100	
Flash Fired		Flash did not fire, compulsory flash mode	
Metering Mode		Pattern	
Exposure Program		Normal program	
Focal Length		85,0 mm	

Slika 3.1: *Primjer EXIF podataka digitalne slike*

možete pronaći popis određenih online aplikacija koje brišu metapodatke sa slika, i popis onih aplikacija koje to ne rade.

Zadatak 1. Desnim klikom miša kliknite na proizvoljnu digitalnu sliku, odaberite *Properties*, pa zatim *Details*. Prikazati će vam se EXIF podaci odabrane slike. Što iz prikazanih podataka možete saznati o slici? Izbrišite EXIF podatke slike klikom na *Remove Properties and Personal Information*. Je li se veličina slike promijenila?

Zadatak 2. Proučite JPEG File Interchange Format standard. Gdje su unutar JPEG datoteke zapisani metapodaci i kako im se može pristupiti?

Zadatak 3. Proučite strukturu EXIF zaglavlja RGB slike te ga ukratko opišite. Napomena: korisne informacije o EXIF zaglavlju nekomprimirane RGB slike možete pronaći u [5, 6].

Budući da se EXIF podaci mogu jednostavno mijenjati (npr. pomoću specijaliziranih softvera o kojima više informacija možete doznati u [7]), ne mogu se uvijek uzeti kao pouzdani dokazi o nekom događaju, već trebaju biti sagledani u okviru dodatnih informacija. Poneki proizvođači fotoaparata i kamera imaju ugrađene načine na koje potpisuju slike, no ni ti potpisi uglavnom nisu potpuno sigurni od eventualnih modifikacija. U [8] se mogu pronaći dodatne informacije o načinima identifikacije digitalnih kamera kojima su slike snimljene, kao što su npr. aberacija leće, nesavršenosti senzora, itd., te o načinima detekcije lažiranih slika.

Primjer dohvaćanja i ispisivanja EXIF informacija o slici u Pythonu:

```
from PIL import Image
from PIL.ExifTags import TAGS
```

```

metaData = {}

slika = Image.open("slika.jpg")
exifPodaci = slika._getexif()

if exifPodaci:
    for (tag, vrijednost) in exifPodaci.items():
        imeTaga = TAGS.get(tag, tag)
        metaData[imeTaga] = vrijednost
        print imeTaga, vrijednost

```

Primjer dohvaćanja i ispisivanja specifične EXIF informacije (u ovom slučaju informacije o aplikaciji kojom se je obrađivala slika):

```

from PIL import Image
from PIL.ExifTags import TAGS

metaData = {}

slika = Image.open("slika.jpg")
exifPodaci = slika._getexif()

# Ime svakog EXIF taga definirano je EXIF standardom
trazeniTag = 'Software'
flag = 0

for (i,j) in exifPodaci.iteritems():
    if TAGS.get(i) == trazeniTag:
        print("Trazimo: " + trazeniTag)
        print j
        flag = 1

if (flag == 0):
    print("Trazimo: " + trazeniTag)
    print ("Trazena vrijednost nije u EXIF podacima slike")

```

Zadatak 4. Učitajte sliku u boji u Pythonu i ispišite njezine GPS koordinate. GPS koordinate zatim promijenite u Pythonu, no pripazite na to da ostali metapodaci ostanu isti kao na ulaznoj slici.

- Ime taga koji se u EXIF standardu koristi za dohvaćanje GPS koordinata na kojima je slika snimljena možete pronaći u [6].
- Slike koje u svom EXIF zaglavlju imaju GPS koordinate možete pronaći na Flickr-u [9], samo pripazite na to da su slike koje želite koristiti pod licencom koja vam dozvoljava njihovo korištenje.

Primjer promjene nadmorske visine u EXIF podacima pomoću biblioteke **piexif**:

```
from PIL import Image
import piexif

slika = Image.open("slika.jpg")
exif_dict = piexif.load(slika.info['exif'])

altitude = exif_dict['GPS'][piexif.GPSIFD.GPSAltitude]
print(altitude)

exif_dict['GPS'][piexif.GPSIFD.GPSAltitude] = (140, 1)

exif_bytes = piexif.dump(exif_dict)

piexif.insert(exif_bytes, 'slika.jpg')
```

4 Provjera autentičnosti digitalne slike

Provjera autentičnosti digitalne slike podrazumijeva potragu za dokazima da slika u pitanju nije mijenjana. Budući da je moguće sliku promijeniti i dokaze o toj promjeni vješto sakriti, najčešće nije moguće u potpunosti isključiti mogućnost da slika nije mijenjana. Ipak, postoje različite metode pomoću kojih se mogu otkriti dokazi da je slika mijenjana, a neke od njih koje se najčešće koriste ćemo upoznati u ovoj laboratorijskoj vježbi.

4.1 Metapodaci

Metapodaci su informacije o digitalnoj slici koji ne uključuju informacije o boji piksela te slike. Oni sadrže informacije o uređaju kojim je slika snimljena te o postavkama tog uređaja prilikom snimanja te slike. Kao što smo vidjeli u prethodnoj laboratorijskoj vježbi, putem metapodataka može se saznati kada je slika mijenjana, gdje je snimljena, kojim je uređajem snimljena, itd. Budući da smo vidjeli i da se metapodaci mogu jednostavno promijeniti, oni nisu najpouzdaniji način za provjeru autentičnosti određene slike.

4.2 Checksum

Checksum (ili ponekad **hash sum**) je podatak (obično prikazan kao niz brojeva i slova) koji jednoznačno identificira neku digitalnu strukturu (npr. sliku ili dokument), slično kao što otisak prsta identificira ljude. U forenzici digitalne slike može se koristiti u svrhu identifikacije te slike (npr. može se provjeriti je li slika mijenjana ili je li koruptirana prilikom prijenosa).

Primjer računanja checksuma (pomoću različitih kriptografskih algoritama) u Pythonu:

```
from PIL import Image
import hashlib

putanja = "slika.jpg"

checksum_md5 = hashlib.md5(open(putanja, 'rb').read()).hexdigest()
checksum_sha1 = hashlib.sha1(open(putanja, 'rb').read()).hexdigest()
checksum_sha224 = hashlib.sha224(open(putanja,
'rb').read()).hexdigest()
checksum_sha256 = hashlib.sha256(open(putanja,
'rb').read()).hexdigest()
checksum_sha384 = hashlib.sha384(open(putanja,
'rb').read()).hexdigest()
```

```
checksum_sha512 = hashlib.sha512(open(putanja,
'rb').read()).hexdigest()

print("MD5 - " + checksum_md5)
print("SHA1 - " + checksum_sha1)
print("SHA224 - " + checksum_sha224)
print("SHA256 - " + checksum_sha256)
print("SHA384 - " + checksum_sha384)
print("SHA512 - " + checksum_sha512)
```

Zadatak 1. Izaberite jedan od kriptografskih algoritama koji se u gornjem primjeru koriste za računanje checksuma i ukratko ga opišite.

Zadatak 2. U Pythonu učitajte sliku i izračunajte njezin checksum. Sliku zatim malo promijenite, sačuvajte, i izračunajte njezin checksum. Usporedite dva dobivena checksuma.

4.3 Informacije o digitalnoj kameri

Postoje različite metode pomoću kojih se podatak o uređaju kojim je snimljena određena digitalna slika može koristiti u analizi autentičnosti te slike u forenzici, a neke od njih su npr. analiza karakteristika leće kamere jer leće produciraju različite aberacije koje ostavljaju jedinstvene tragove na slici, te analiza nesavršenosti senzora kamere [10].

U ovome poglavlju mi ćemo se usredotočiti na provjeravanje specifikacija kamere, te otkrivanje PRNU otiska kamere.

4.3.1 Provjera specifikacija kamere

Budući da ne podržavaju sve kamere sve postojeće formate i rezolucije za zapis digitalnih slika, iz formata i rezolucije u kojima se određena slika nalazi može biti moguće saznati je li promijenjena. Na primjer, pretpostavimo da smo iz metapodataka određene slike *A* dobili informaciju o kameri *B* kojom je slika *A* snimljena. Ako se slika *A* nalazi u formatu ili rezoluciji *C* koju kamera *B* ne podržava, onda možemo pretpostaviti da se slika *A* ne nalazi u svom izvornom obliku.

4.3.2 PRNU otisak kamere

PRNU (engl. *Photo Response Non Uniformity*) otisak digitalne kamere je rezultat nesavršenosti senzora te kamere i može se koristiti u svrhu identificiranja uređaja kojim je digitalna fotografija snimljena budući da je uglavnom jedinstven za svaku kameru. PRNU otisak ne bi postojao da

su senzori digitalnih kamera savršeni, tj. da za jednake intenzitete svjetla koji padnu na senzor uvijek zapisuju jednake brojeve koji odgovaraju tom intenzitetu. Uzrok nesavršenosti senzora može biti materijal od kojega je senzor izgrađen, vrijeme ekspozicije slike, temperatura, itd. Estimacija digitalnog otiska nekog uređaja može se dobiti kao razlika originalne slike i originalne slike na koju je primijenjen filter koji uklanja šum sa slike (engl. *denoising filter*) [11].

Zadatak 3. Proučite i ukratko opišite najpoznatije filtere za uklanjanje šuma sa digitalne slike. Neki od njih u Pythonu primijenite na sliku i usporedite dobiveni rezultat s originalnom slikom.

Zadatak 4. U Pythonu-u napišite funkciju za određivanje digitalnog otiska uređaja kojim je snimljena digitalna slika.

Zapamtite da se po PNRU uzorku može prepoznati uređaj kojim je snimljena neka slika, a po tom uzorku se mogu pronaći i druge slike snimljene istim uređajem. Sve ovo na kraju može dovesti do identifikacije osobe koja je slike snimala, stoga pripazite što sve stavljate na Internet. Točan digitalni otisak svojih kamera nemojte stavljati na Internet jer bi se mogli zlouporabiti.

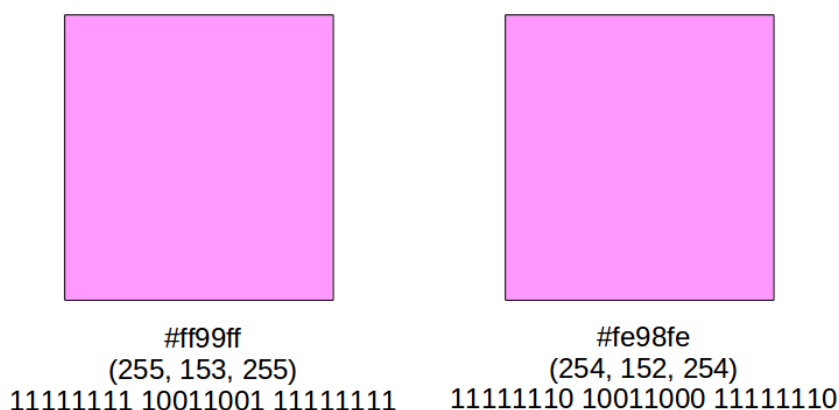
Zadatak 5. U Python učitajte nekoliko različitih slika i pokušajte odrediti jesu li neke od njih snimljene istim uređajem.

5 Steganografija

Steganografija se u obradi i analizi digitalne slike koristi za prikrivanje informacija, te je stoga važna u forenzičkoj analizi digitalne slike.

Steganografija (engl. *steganography*) podrazumijeva skrivanje podataka unutar nekih drugih podataka. Dolazi od grčkih riječi *steganos* (zaštićen ili skriven) i *graphein* (pisanje) [12]. Primjer tradicionalne starinske steganografije bio bi pisanje poruka limunovim sokom na praznim listovima papira. Te bi se poruke onda mogle pročitati na način da se ispod papira drži upaljen plamen jer bi slova tada postala vidljiva. U digitalnom dobu ovaj starinski način steganografije se više uglavnom ne koristi, jer ga je zamijenila digitalna steganografija.

Primjeri digitalne steganografije su raznovrsni, a jedan od najpoznatijih je skrivanje informacija u 24-bitne slikovne datoteke. U ovom primjeru steganografije u najmanje značajne bitove piksela na slici skrivaju se ASCII znakovi, i na taj način se prenose tajne poruke. Kada se najmanje značajni bit nekog piksela promijeni, sama boja piksela se neće značajno promijeniti (pogledati sliku 5.1) te tajna poruka neće vidljivo utjecati na prikaz digitalne slike. Više informacija o primjenama digitalne steganografije možete pronaći u [12].



Slika 5.1: *Primjer promjene boje nakon promjene najmanje značajnih bitova svakog kanala koji pripada RGB pikselu*

Zadatak 1. Navedite nekoliko primjena digitalne steganografije.

Zadatak 2. Što mislite koje su prednosti a koji nedostaci steganografije u odnosu na enkripciju?

U nastavku ovoga paragrafa dati ćemo primjer digitalne steganografije u kojemu se dva najmanje značajna bita jedne slike (one u koju želimo sakriti podatke) zamjenjuju sa dva najznačajnija bita druge slike (one koju želimo sakriti). Primjer slike u koju želimo sakriti podatke

dan je na slici 5.2, a primjer slike koju želimo sakriti dan je na slici 5.3. Za svaku od ovih slika dani su i odgovarajući binarni zapisi. Na slici 5.4 možete vidjeti primjer zamjene najmanje značajnih bitova sa slike 5.2 sa najznačajnijim bitovima sa slike 5.3. Na slici 5.5 dan je prikaz slike u koju smo željeli sakriti tajne podatke, te prikaz te slike nakon što smo u nju sakrili podatke. Možete primijetiti da se vrijednosti piksela u tim slikama značajno ne razlikuju, što znači da će slika nakon što smo u nju sakrili tajne podatke biti gotovo jednaka slici prije no što smo u nju sakrili tajne podatke.

200	0	255	13	2	1100 1000	0000 0000	1111 1111	0000 1101	0000 0010
42	153	147	255	11	0010 1010	1001 1001	1001 0011	1111 1111	0000 1011
0	100	255	255	0	0000 0000	0110 0100	1111 1111	1111 1111	0000 0000
112	90	17	56	3	0111 0000	0101 1010	0001 0001	0011 1000	0000 0011
111	1	153	147	0	0110 1111	0000 0001	1001 1001	1001 0011	0000 0000

Slika 5.2: Primjer slike u koju želimo sakriti neku drugu sliku i njezin binarni zapis

0	100	0	30	0	0000 0000	0110 0100	0000 0000	0001 1110	0000 0000
11	1	17	1	1	0000 1011	0000 0001	0001 0001	0000 0001	0000 0001
0	40	255	255	0	0000 0000	0010 1000	1111 1111	1111 1111	0000 0000
255	0	17	56	255	1111 1111	0000 0000	0001 0001	0011 1000	1111 1111
255	1	153	255	255	1111 1111	0000 0001	1001 1001	1111 1111	1111 1111

Slika 5.3: Primjer slike koju želimo sakriti u neku drugu sliku i njezin binarni zapis

Zapamtite da sliku u koju ste sakrili neku drugu sliku morate sačuvati u formatu koji ne vrši sažimanje podataka (tzv. sažimanje bez gubitaka (engl. *lossless compression*)), jer bi inače tajna slika mogla biti nečitljiva.

Zadatak 3. U Pythonu napišite funkciju za skrivanje (i kasnije otkrivanje) jedne slike unutar neke druge slike.

- Neka slike u ovome zadatku budu jednakih dimenzija.
- Tajnu sliku sakrijte na način da u dva najmanje značajna bita svih piksela u svim kanalima originalne slike zapisujete vrijednosti dvaju najznačajnijih piksela tajne slike.
- Možete koristiti Pythonove funkcije iz biblioteke *Steganography* za pretvaranje brojeva iz binarnog oblika u dekadski oblik i obrnuto.

Slika u koju želimo sakriti neku drugu sliku:

1100 1000	0000 0000	1111 1111	0000 1101	0000 0010
0010 1010	1001 1001	1001 0011	1111 1111	0000 1011
0000 0000	0110 0100	1111 1111	1111 1111	0000 0000
0111 0000	0101 1010	0001 0001	0011 1000	0000 0011
0110 1111	0000 0001	1001 1001	1001 0011	0000 0000

Slika koju želimo sakriti:

0000 0000	0110 0100	0000 0000	0001 1110	0000 0000
0000 1011	0000 0001	0001 0001	0000 0001	0000 0001
0000 0000	0010 1000	1111 1111	1111 1111	0000 0000
1111 1111	0000 0000	0001 0001	0011 1000	1111 1111
1111 1111	0000 0001	1001 1001	1111 1111	1111 1111

Slika u kojoj je sakrivena druga slika:

1100 1000	0000 0001	1111 1100	0000 1100	0000 0000
0010 1000	1001 1000	1001 0000	1111 1100	0000 1000
0000 0000	0110 0100	1111 1111	1111 1111	0000 0000
0111 0011	0101 1000	0001 0000	0011 1000	0000 0011
0110 1111	0000 0000	1001 1010	1001 0011	0000 0011

Slika 5.4: Primjer digitalne steganografije

200	0	255	13	2
42	153	147	255	11
0	100	255	255	0
112	90	17	56	3
111	1	153	147	0



200	1	252	12	0
40	152	144	252	8
0	100	255	255	0
115	88	16	56	3
111	0	154	147	3

Slika 5.5: Početna slika (lijevo) i slika nakon što su u njoj sakriveni podaci (desno).
Primijetite da nema značajne razlike u vrijednostima piksela na ovim slikama

Zadatak 4. Modificirajte programski kod iz prethodnog zadatka na način da se za skrivanje tajne slike u ulaznu sliku koriste tri bita, a ne dva. Usporedite rezultate s onima koje ste dobili u prethodnom zadatku.

5.0.1 Implementacija steganografije u programskom jeziku Python

U ovome dijelu teksta biti će dani savjeti za implementaciju steganografije u programskom jeziku Python.

Učitavanje slike:

```
slika = Image.open("slika.png")
```

Pokazivač na sliku:

```
matrica_sa_pikselima = slika.load()
```

Dimenzije slike:

```
stupci, retci = slika.size
```

Provjera radi li se o slici u boji:

```
if (slika.mode == "RGB"):
    print("Broj kanala: 3")
```

Ispis vrijednosti piksela slike (za RGB slike ispisuje se vrijednost piksela u obliku: (x, y, z)):

```
print("Vrijednost piksela: " + str(matrica_sa_pikselima[i,j]))
```

Vrijednost piksela u različitim kanalima boje:

```
r = matrica_sa_pikselima[i,j][0]
g = matrica_sa_pikselima[i,j][1]
b = matrica_sa_pikselima[i,j][2]

print("1. kanal: " + str(r))
print("2. kanal: " + str(g))
print("3. kanal: " + str(b))
```

Pretvorba vrijednosti piksela u binarni oblik:

```
r_binary = format(r, '#010b').replace("0b", "")
```

Stvaranje praznog niza:

```
novi_niz = [0, 0, 0, 0, 0, 0, 0, 0]
```

Pretvorba niza koji se sastoji od nula i jedinica u binarni broj:

```
novi_broj = int("".join(map(str, novi_niz)),2)
```

Promjena vrijednosti piksela u RGB prostoru boja:

```
matrica_sa_pikselima[i,j] = (novi_r, novi_g, novi_b)
```

6 Detekcija krivotvorenih JPEG slika pomoću DCT transformacije

Direktna kosinusna transformacija (engl. *Direct Cosine Transform, DCT*) pretvara sliku ili signal iz prostorne u frekvencijsku domenu. Za većinu slika vrijedi da im se energija nalazi u nižim frekvencijama, a to su one koje odgovaraju gornjem lijevom kutu DCT matrice. Ako zadržite samo nekoliko prvih energija a ostale odbacite, to će rezultirati sažimanjem slike. Na primjer, u JPEG kompresiji slika se dijeli na blokove od 8x8 piksela, i za svaki blok se računa DCT. DCT je slična brzom Fourierovoj transformaciji, a razlikuje se u tome što može dobro aproksimirati linije sa manje koeficijenata.

Formula za računanje DCT-a [13]:

$$DCT(i, j) = \frac{C(i) \cdot C(j)}{\sqrt{2N}} \cdot \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} pixel(x, y) \cdot \cos \frac{(2x+1) \cdot i \cdot \pi}{2N} \cdot \cos \frac{(2y+1) \cdot j \cdot \pi}{2N} \quad (6.1)$$

$$C(x) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{ako je } x = 0 \\ 1, & \text{ako je } x > 0 \end{cases} \quad (6.2)$$

U formulama 6.1 i 6.2 vrijedi slijedeće:

- (i, j) – koordinate piksela u bloku za kojega se računa DCT,
- $N \times N$ – dimenzije bloka slike za kojeg se računa DCT (najčešće 8x8 piksela),
- (x, y) – koordinate trenutnog piksela u bloku,

Primjer koda za učitavanje slike i njezinu pretvorbu u sliku u razinama sive boje, podjelu slike na blokove veličine 8x8 piksela te računanje DCT matrice za svaki od tih blokova:

```
from PIL import Image
import numpy as np
from scipy.fftpack import dct, idct

slika = Image.open("dct_test3.png")
slika_grayscale = slika.convert('L')
slika_matrix = np.asarray(slika_grayscale)
```

```

velicina_bloka = 8
stupci, retci = slika_grayscale.size

broj_blokova_1 = retci / 8
broj_blokova_2 = stupci / 8

print("broj_blokova_1 = " + str(broj_blokova_1))
print("broj_blokova_2 = " + str(broj_blokova_2))
print("\n")

trenutni_pocetni_stupac = 0
trenutni_pocetni_redak = -velicina_bloka

for redak in range(broj_blokova_1):
    trenutni_pocetni_stupac = 0
    trenutni_pocetni_redak = trenutni_pocetni_redak + velicina_bloka
    for stupac in range(broj_blokova_2):
        temp_matrix = np.zeros(shape = (velicina_bloka,
velicina_bloka))
        temp_matrix_dct = np.zeros(shape = (velicina_bloka,
velicina_bloka))

        trenutni_redak = 0
        for i in range (trenutni_pocetni_redak,
trenutni_pocetni_redak + velicina_bloka):
            trenutni_stupac = 0
            for j in range (trenutni_pocetni_stupac,
trenutni_pocetni_stupac + velicina_bloka):
                temp_matrix[trenutni_redak, trenutni_stupac] =
slika_matrix[i,j]
                trenutni_stupac = trenutni_stupac + 1
                trenutni_redak = trenutni_redak + 1

            print("Trenutni blok: ")
            print(temp_matrix)
            print("\n")

            temp_matrix_dct = dct(dct(temp_matrix.T).T)

            print("DCT matrica za trenutni blok: ")
            print(temp_matrix_dct)
            print("\n")

            trenutni_pocetni_stupac = trenutni_pocetni_stupac +
velicina_bloka

print("Kraj programa!\n")

```

Kada se za blokove slike izračunaju DCT matrice, gornji lijevi element DCT matrice određenog bloka može se uspoređivati s gornjim lijevim elementima ostalih blokova slike da bi se odredilo koji mu je blok najbliži. Što su gornji lijevi elementi DCT matrica određenih blokova bliži, to su sami blokovi sličniji.

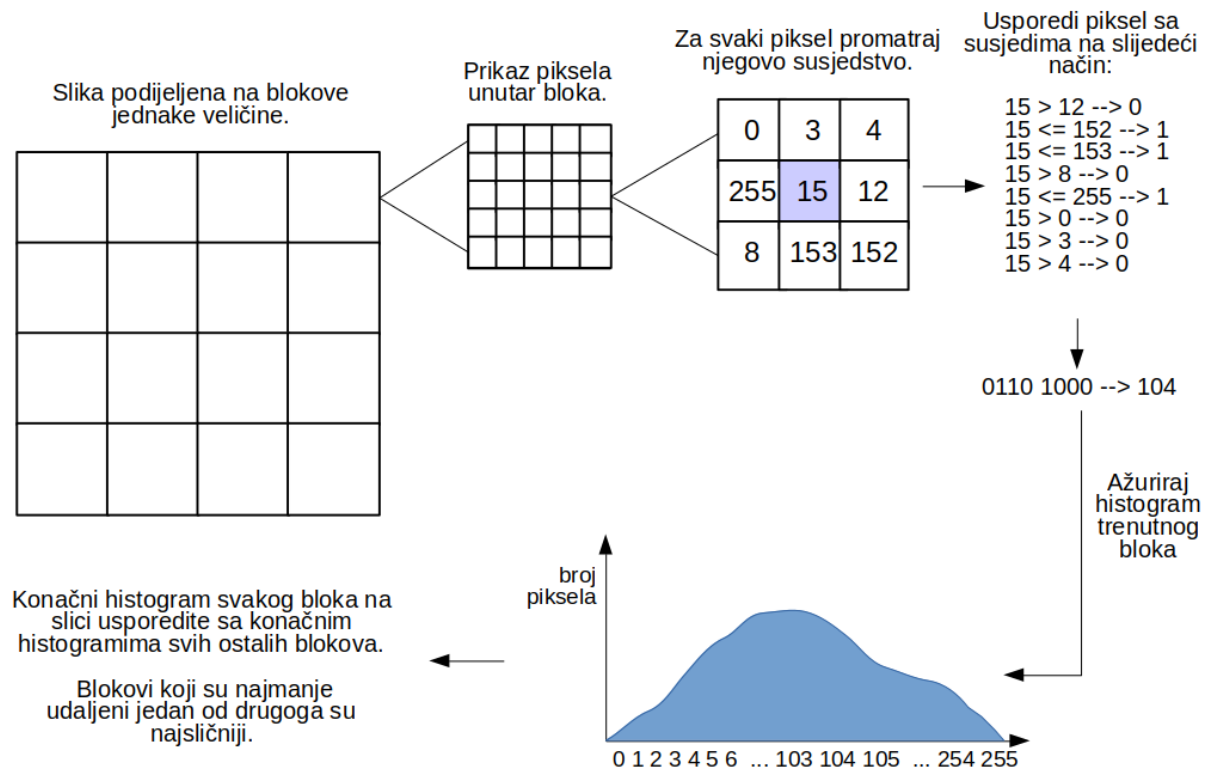
Zadatak 1. Odaberite proizvoljnu digitalnu sliku te ju u nekom od programa za obradu slike obradite na način da duplicirate nekoliko njezinih blokova. Sliku spremite te ju učitajte u Python i konvertirajte u sliku u razinama sive boje.

Zadatak 2. Pomoću DCT transformacije pronađite slične blokove slike učitane u prethodnom zadatku. Koristite **nepreklapajuće** blokove različitih veličina te odredite koja veličina bloka daje najbolje rezultate.

Zadatak 3. Pomoću DCT transformacije pronađite slične blokove slike učitane u prethodnom zadatku. Koristite **preklapajuće** blokove različitih veličina te odredite koja veličina bloka daje najbolje rezultate.

7 Detekcija krivotvorenih JPEG slika pomoću LBP algoritma

Detekcija kloniranih uzoraka na digitalnoj slici može se, uz DCT transformaciju, otkriti i pomoću LBP (engl. *Local Binary Patterns*) [14, 15] algoritma. Prikaz detekcije uzoraka na slici pomoću ovoga algoritma dan je na slici 7.1.



Slika 7.1: Detekcija uzoraka na slici pomoću LBP algoritma

Zadatak 1. Istražite na koje se sve načine može odrediti udaljenost između dva niza, te koji su od njih implementirani u Pythonu. Kreirajte dva niza u Pythonu te odredite njihovu međusobnu udaljenost.

Zadatak 2. Odaberite proizvoljnu digitalnu sliku te ju u nekom od programa za obradu slike obradite na način da duplirate nekoliko njezinih blokova. Sliku spremite te ju učitajte u Python i konvertirajte u sliku u razinama sive boje.

Zadatak 3. Na slici učitanoj u drugom zadatku pokušajte pronaći duplirane blokove na način prikazan slikom 7.1. Udaljenost između histograma izračunajte pomoću funkcije koju ste koristili u prvom zadatku.

8 Antiforenzika

Antiforenzika digitalne slike (engl. *image antiforensics*, *image counterforensics*) obuhvaća različite tehnike koje se mogu koristiti prilikom postupka modifikacije digitalne slike i skrivanja svih tragova te modifikacije. Antiforenziku digitalne slike najčešće provode osobe sa dubokim znanjem forenzike digitalne slike, jer znaju na koje stvari trebaju obratiti pozornost da bi forenzičarima otežali ili onemogućili posao.

Zadatak 1. Koristeći se saznanjima iz prethodnih laboratorijskih vježbi, pokušajte modificirati proizvoljnu digitalnu sliku i sakriti sve tragove te modifikacije.

Zadatak 2. Koristeći se saznanjima iz prethodnih laboratorijskih vježbi, pokušajte napisati program koji bi mogao otkriti je li određena slika modificirana ili ne. Budući da postoji mnoštvo načina na koje se slika može modificirati, usredotočite se na otkrivanje jedne od tih metoda, a ne svih.

Literatura

- [1] Pablo Arbeláez, Michael Maire, Charless Fowlkes, and Jitendra Malik. Contour detection and hierarchical image segmentation. *IEEE TPAMI*, 33:898–916, May 2011.
- [2] Jonathan Bailey. How to ensure your watermarks are secure. <https://www.plagiarismtoday.com/2013/08/01/how-to-ensure-your-watermarks-are-secure/>, 2013. Accessed: 2018-09-10.
- [3] Digital Image Integrity. Link: https://www.adobe.com/digitalimag/pdfs/digital_image_integrity.pdf, posljednji put pristupljeno: 31. listopada 2017.
- [4] Igor Kuksov. Do your online photos respect your privacy?, October 31st, 2016. Link: <https://www.kaspersky.com/blog/exif-privacy/13356/>, posljednji put pristupljeno: 03. studenog 2017.
- [5] Standardization Committee, Standard of the Camera & Imaging Products Association. Exchangeable image file format for digital still cameras: Exif Version 2.3. http://www.cipa.jp/std/documents/e/DC-008-2012_E.pdf, December 2012. Accessed: 2018-09-04.
- [6] Phil Harvey. EXIF Tags. <https://www.sno.phy.queensu.ca/~phil/exiftool/TagNames/EXIF.html>. Accessed: 2018-09-05.
- [7] Comparison of metadata editors. Link: https://en.wikipedia.org/wiki/Comparison_of_metadata_editors, posljednji put pristupljeno: 03. studenog 2017.
- [8] Tran Van Lanh, Kai-Sen Chong, Sabu Emmanuel, and Mohan S. Kankanhalli. A survey on digital camera image forensic methods. *Proceedings of the 2007 IEEE International Conference on Multimedia and Expo (ICME 2007)*, pages 16–19, 2007.
- [9] Flickr. <https://www.flickr.com/>. Accessed: 2018-09-04.
- [10] Alessandro Piva. An overview on image forensics. *ISRN Signal Processing*, 2013.
- [11] V. Connoter and G. Boato. Analysis of sensor fingerprint for source camera identification. *Electronics Letters*, 47, 2011.
- [12] Steganography. Link: <https://en.wikipedia.org/wiki/Steganography>, posljednji put pristupljeno: 06. studenog 2017.

- [13] Eric Roberts. The Discrete Cosine Transform (DCT). <https://cs.stanford.edu/people/eroberts/courses/soco/projects/data-compression/lossy/jpeg/dct.htm>. Accessed: 2018-09-12.
- [14] D. C. He and L. Wang. Texture unit, texture spectrum, and texture analysis. *IEEE Transactions on Geoscience and Remote Sensing*, 28:509–512, 1990.
- [15] L. Wang and D. C. He. Texture classification using texture spectrum. *Pattern Recognition*, 23:905–910–512, 1990.